

prof算子耗时占比分析

1.模型启动与prof抓取

环境设置

建议采用最新镜像进行prof分析<http://image.sourcefind.cn:5000/dcu/admin/base/vllm:0.9.2-ubuntu22.04-dtk25.04.1-rc5-rocblas104381-0915-das1.6-py3.10-20250916-rc2>

以vllm-092为例，根据镜像<http://image.sourcefind.cn:5000/dcu/admin/base/vllm:0.9.2-ubuntu22.04-dtk25.04.1-rc5-rocblas104381-0915-das1.6-py3.10-20250916-rc2> 创建容器，环境变量如下所示：

在相关可运行的软件环境下抓取，可在~/.bashrc中添加如下环境变量

代码块

```
1 export VLLM_TORCH_PROFILER_DIR=/workspace/int8prof/  
2 export DEBUG_CLR_GRAPH_PACKET_CAPTURE=false
```

其中，`VLLM_TORCH_PROFILER_DIR` 是保存的prof文件路径，`DEBUG_CLR_GRAPH_PACKET_CAPTURE=false` 是防止prof抓取算子出错，必须加上

模型启动

以deepseek-r1-w4a8-v2为例，用如下命令行启动模型

代码块

```
1 nohup vllm serve /nvme1/models/DeepSeek-R1-0528-W4A8-V2/ -q  
  slimquant_w4a8_marlin --trust-remote-code --dtype bfloat16 --max-model-len  
  5000 --max-seq-len-to-capture 5000 -tp 8 --gpu-memory-utilization 0.92 --max-  
  num-seqs 256 --disable-log-requests --block-size 64 --no-enable-chunked-  
  prefill --max-num-batched-tokens 5000 --no-enable-prefix-caching --compilation-  
  config '{"full_cuda_graph": true}' > /workspace/logs/092_0812/test.log 2>&1 &
```

启动完成后，简单对话一下看是否顺利启动

代码块

```

1 curl http://0.0.0.0:8000/v1/chat/completions -H "Content-Type: application/json" -d '{
2   "model": "/nvme1/models/DeepSeek-R1-0528-W4A8-V2/",
3   "messages": [{"role": "user", "content": "你好，帮我写一个贪吃蛇"}],
4   "temperature":0,
5   "max_tokens":200
6 }'

```

```

root@b41:workspace# curl http://0.0.0.0:8000/v1/chat/completions -H "Content-Type: application/json" -d '{
  "model": "/nvme1/models/DeepSeek-R1-0528-W4A8-V2/",
  "messages": [{"role": "user", "content": "你好，帮我写一个贪吃蛇"}],
  "temperature":0,
  "max_tokens":200
}'
{"id":"chatcmpl-9a3bdf7e40244fda736f8a2d168f5f","object":"chat.completion","created":1760000666,"model":"/nvme1/models/DeepSeek-R1-0528-W4A8-V2/","choices":[{"index":0,"message":{"role":"assistant","reasoning_content":"","content":"\n我们将使用Pygame来创建一个简单的贪吃蛇游戏。游戏包括以下功能：\n1. 蛇的移动（通过方向键控制）\n2. 食物的生成（随机位置）\n3. 吃到食物后蛇身变长\n4. 游戏结束条件（碰到边界或自身）\n\n\n步骤\n1. 初始化pygame\n2. 设置游戏窗口大小和标题\n3. 定义颜色\n4. 设置蛇的初始位置和长度（用一个列表表示，每个元素是一个方块的位置）\n5. 设置食物的初始位置\n6. 设置初始移动方向\n7. 游戏主循环：\n   a. 处理事件（特别是退出事件和方向键事件）\n   b. 根据方向移动蛇头，并更新蛇身\n   c. 检查是否吃到食物，如果吃到，蛇身增长，并在新位置生成食物\n   d. 检查是否碰到边界或自身\n\n\n工具调用\n无\n"},"tool_calls":[],"logprobs":null,"finish_reason":"length","stop_reason":null}

```

如上所示对话正常

抓取prof

用如下命令抓取prof，抓取的prof文件在上述的 `VLLM_TORCH_PROFILER_DIR` 保存路径中

代码块

```

1 vllm bench serve --model /nvme1/models/DeepSeek-R1-0528-W4A8-V2/ --random-
  input-len 4096 --random-output-len 20 --num-prompts 21 --dataset-name random --
  ignore-eos --profile

```

抓取输入4k，输出20，21并发的prof，由于是8卡，在保存路径下生成8个pt文件，选一个下载到本地

注意点：

1. 建议输出不用太长，否则prof文件会很大，20即可；
2. 对于抓取多少并发对应的prof文件，根据测试表格数据，选择感兴趣的并发数进行抓即可，本例以21并发为例；

```

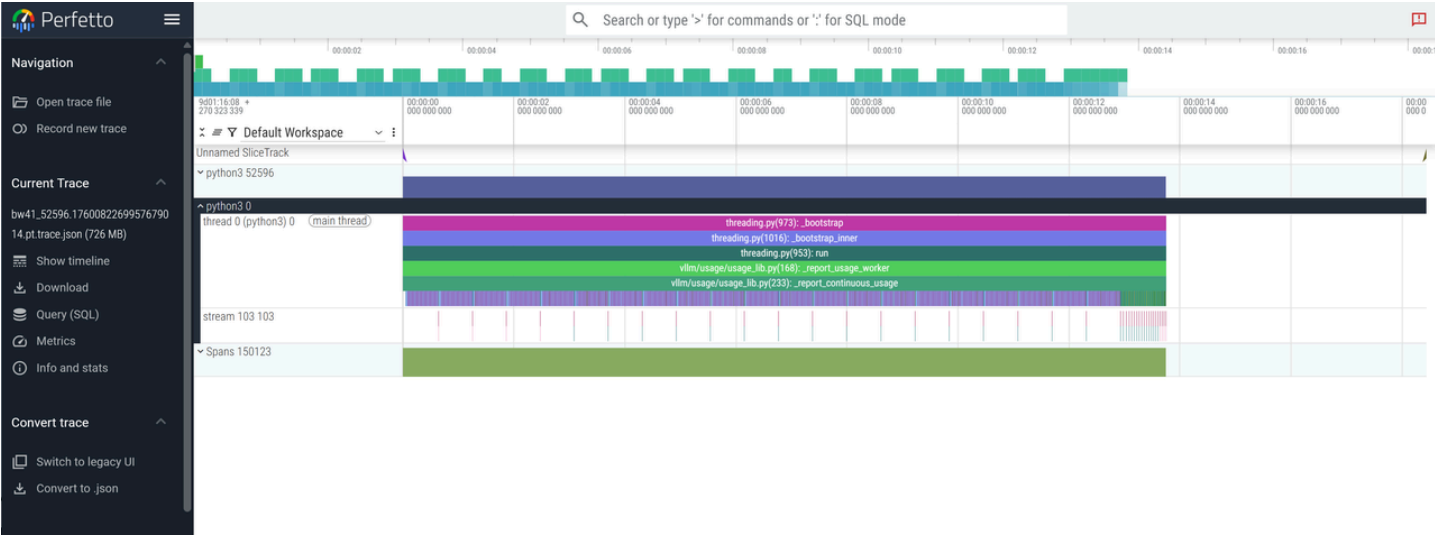
root@b41:workspace# vllm bench serve --model /nvme1/models/DeepSeek-R1-0528-W4A8-V2/ --random-input-len 4096 --random-output-len 20 --num-prompts 21 --dataset-name random --ignore-eos --profile
INFO 10-10 15:43:54 [__init__.py:245] Automatically detected platform rocm.
WARNING 10-10 15:43:58 [api_server.py:106] Torch Profiler is enabled in the API server. This should ONLY be used for local development!
NameSpace[server:bench, bench_type=serve, dispatch_function=function BenchmarkServingSubcommand.cmd at 0x7f13e64fbeb0, seed=0, num_prompts=21, dataset_name=random, dataset_path=None, custom_output_len=256, custom_skip_chat_template=False, sonnet_input_len=550, sonnet_output_len=100, sonnet_prefix_len=200, sharegpt_output_len=None, random_input_len=4096, random_output_len=20, random_range_ratio=0.0, random_prefix_len=0, hf_subset=None, hf_split=None, hf_output_len=None, endpoint_type=openai, label=None, backend=vllm, base_url=None, host=127.0.0.1, port=8000, endpoint=/v1/completions, max_concurrency=None, model=/nvme1/models/DeepSeek-R1-0528-W4A8-V2/, tokenizer=None, use_beam_search=False, logprobs=None, request_rate=1.0, burstiness=1.0, trust_remote_code=False, disable_torch_profiler=False, profile=True, save_result=False, save_details=False, append_result=False, metadata=None, result_dir=None, result_filename=None, ignore_eos=True, percentile_metrics=tft, tpot, ttl, metric_percentiles=99, goodput=None, top_p=None, top_k=None, min_p=None, temperature=None, tokenizer_mode=auto, served_model_name=None, lora_model=None, ram_usage_strategy=None, ramp_up_start=None, ramp_up_end=None]
INFO 10-10 15:43:59 [datasets.py:355] Sampling input len from [4095, 4095] and output len from [20, 20]
Starting initial single prompt test run...
Initial test run completed. Starting main benchmark run...
Starting profiler...
Traffic request rate: inf
Durability factor: 1.0 (Poisson process)
Maximum request concurrency: None
90%
Stopping profiler...
100%
===== Serving Benchmark Result =====
Successful requests: 21
Benchmark duration (s): 121.09
Total input tokens: 8595
Total generated tokens: 420
Request throughput (req/s): 0.17
Output token throughput (tok/s): 3.47
Total token throughput (tok/s): 733.62
-----Time to First Token-----
Mean TTFT (ms): 6763.62
Median TTFT (ms): 6751.84
P99 TTFT (ms): 12756.60
-----Time per Output Token (excl. 1st token)-----
Mean TPOUT (ms): 344.22
Median TPOUT (ms): 346.85
P99 TPOUT (ms): 614.43
-----Inter-token Latency-----
Mean ITL (ms): 344.22
Median ITL (ms): 611.26
P99 ITL (ms): 800.72
root@b41:workspace# cd /ntb/ntb/
root@b41:workspace# ls
bw1_52596.1760002269957679014.pt.trace.json.gz  bw1_52598.1760002270496034670.pt.trace.json.gz  bw1_52600.17600022686222020831.pt.trace.json.gz  bw1_52602.1760002268841050282.pt.trace.json.gz
bw1_52597.1760002270101506110.pt.trace.json.gz  bw1_52599.176000227016693354.pt.trace.json.gz  bw1_52601.176000227018346571.pt.trace.json.gz  bw1_52603.1760002268024366198.pt.trace.json.gz

```

2.prof分析及算子耗时占比分析

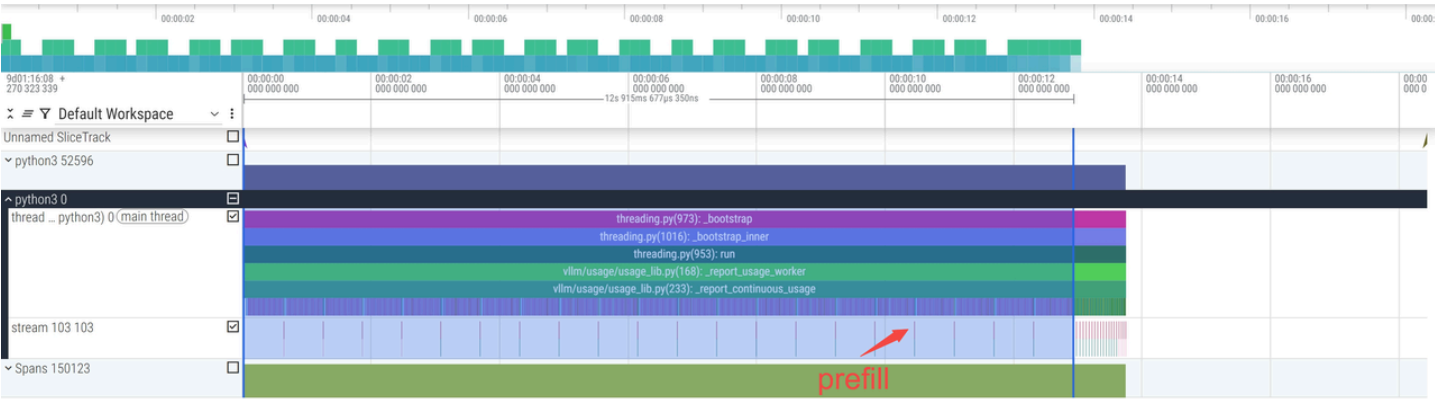
prof分析

将抓取的pt文件下载到本地并解压，在浏览器打开<https://ui.perfetto.dev/>，将解压完成的json文件拖入，如下所示

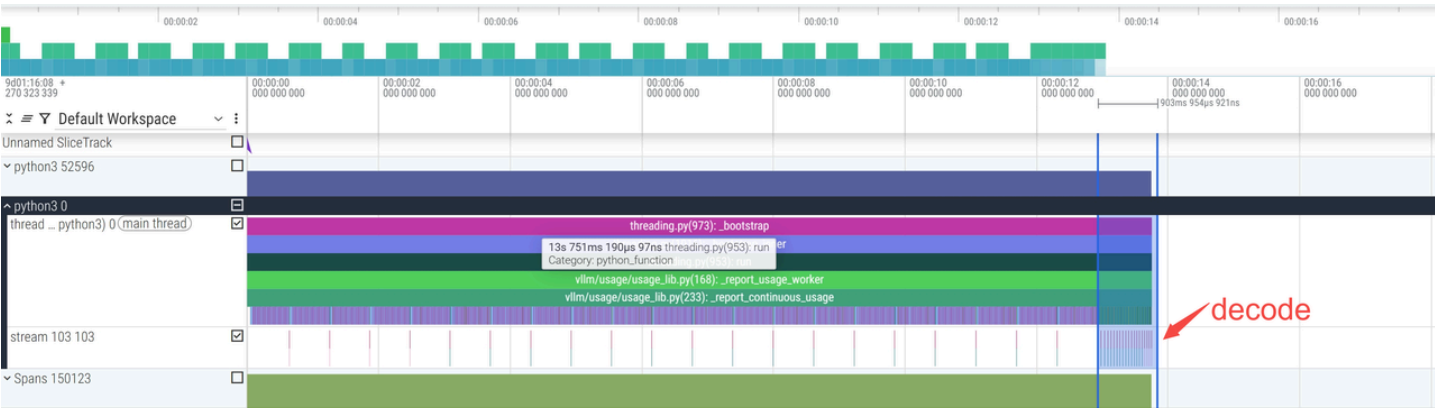


prefill和decode阶段划分

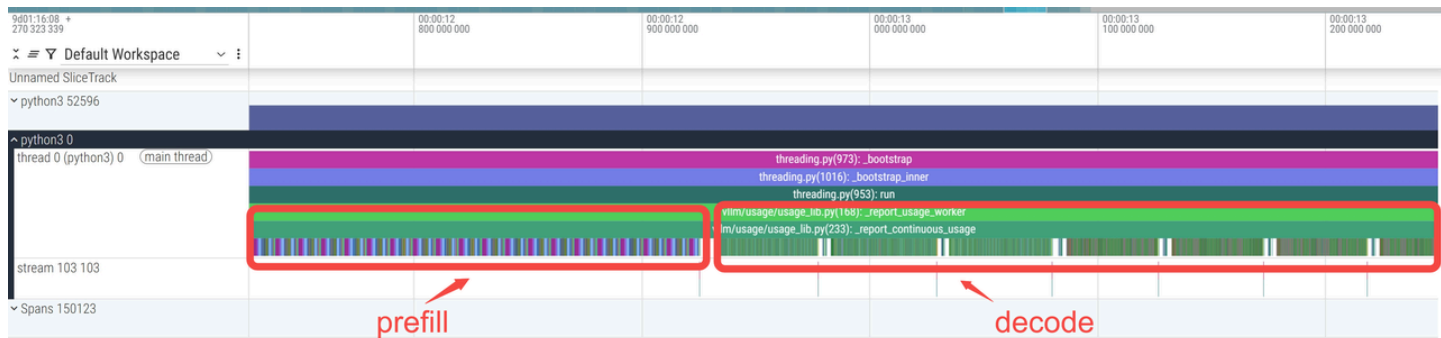
如下所示，前面一段紫色的为prefill阶段，prefill阶段的ncclKernel_AllGather_Ring_Simple_Sum__i8算子较为稀疏



如下所示，后面一段绿色的为decode阶段，decode阶段的ncclKernel_AllGather_Ring_Simple_Sum__i8算子较为密集



prefill和decode阶段有明显的间隙和颜色划分，可用鼠标拉取阶段，并用“wasd”键控制大小和方向进行算子的查看



常见算子说明

1. 激活量化算子与量化gemm算子

BW算子名称



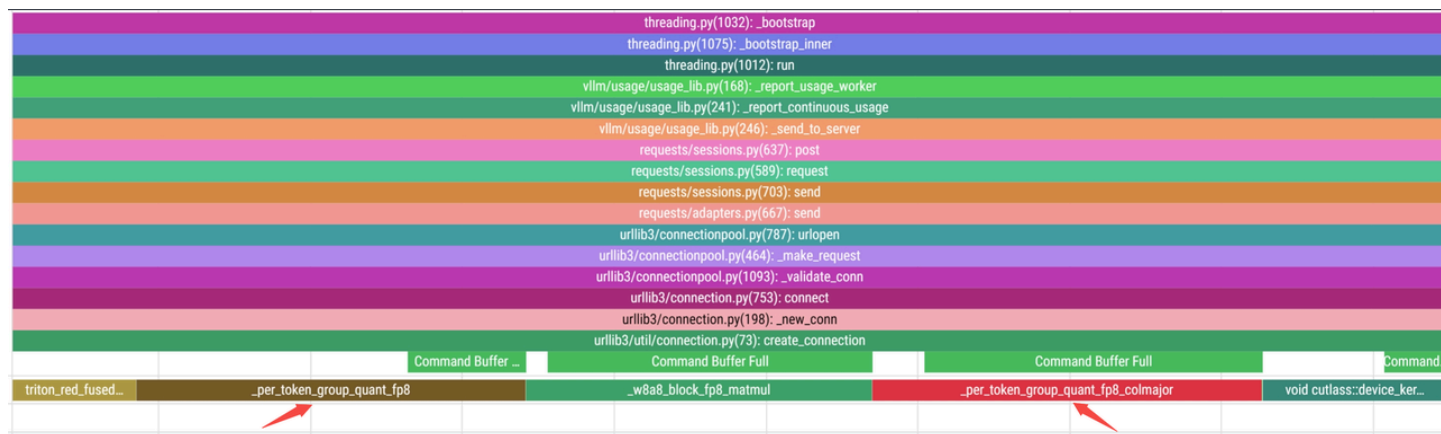
_per_token_quant_int8为激活量化算子，将输入的bf16量化为int8，常用于W8A8量化中；

以“Cijk”开头并以“Squant”结尾的为量化的int8类型的gemm算子，常用于W8A8量化中；

在统计中，可以将以“Cijk”开头并以“Squant”结尾的算子归结于一类算子，为int8类型的gemm算子；

gemm_w8_a8 字眼的也是量化gemm算子。

H20中



以fp8算子为例，以上两个都是为fp8的激活量化算子，有per_token_group_quant字眼。



名字中带“cutlass_3x_gemm_fp8_blockwise”和“_w8a8_block_fp8_matmul”的对应BW中的Cijk_..._Squant算子，负责计算int8的量化gemm

量化gemm算子在非MOE量化模型中耗时占比会较高，是重点分析的热点算子。

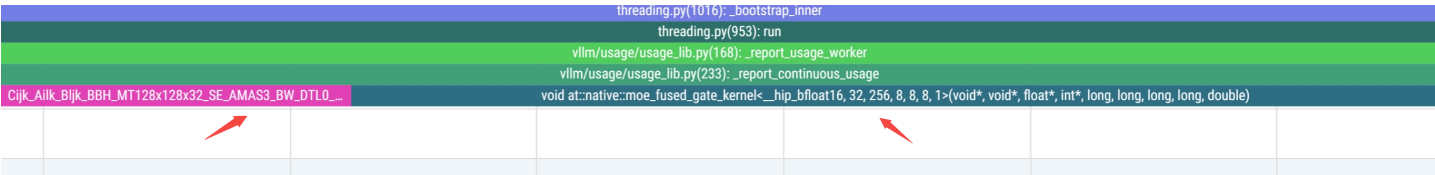
2.pa /fa /mla算子

名字中带“mla”与“FlashAttnFwdSm90”的为mla/pa/fa的相关算子

在decode阶段，随着并发的增加pa占比(ds中是mla) 会增加，耗时占比高，是重点分析的热点算子，在prefill阶段分析fa占比；

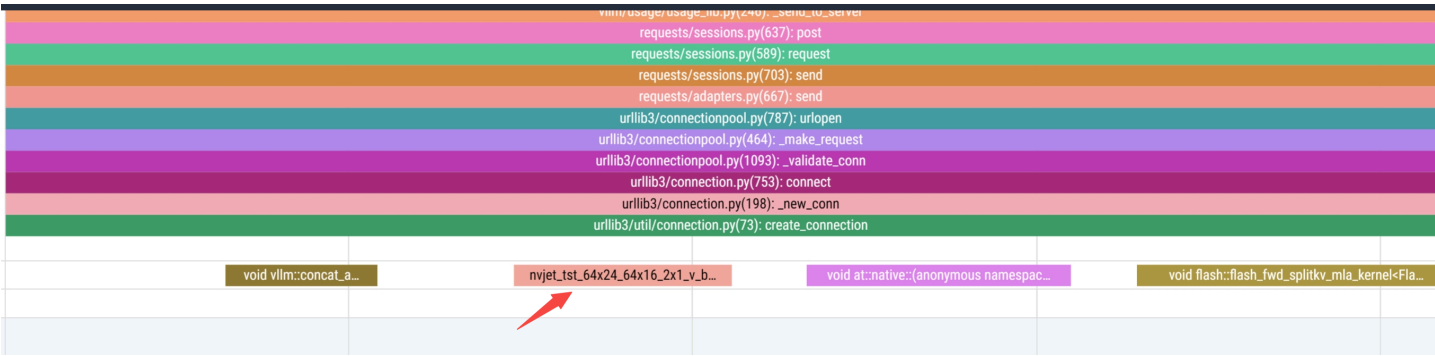
3.普通fp16 gemm算子

BW算子名称



以“Cijk”开头并不以“Squant”结尾的为bf16类型的gemm算子

H20中



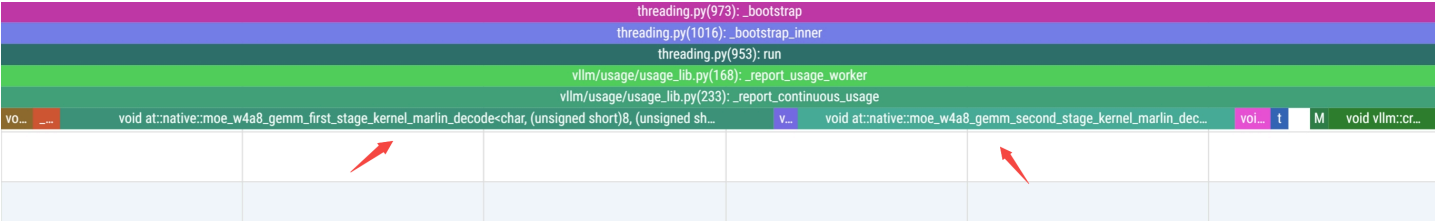
名字中带“nvjet”的为bf16类型的gemm算子

在量化模型中，fp16 gemm的占比没有那么多高，通常在5%以下。

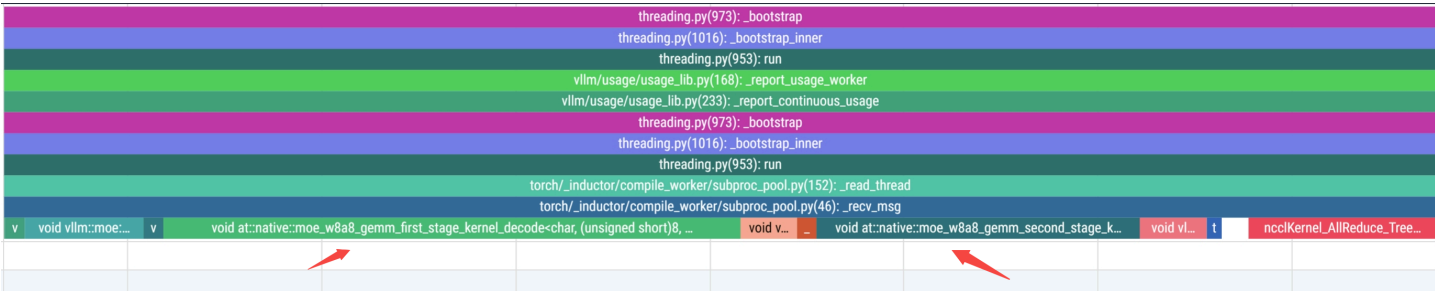
4.MOE算子

BW算子名称

在量化的MOE模型中，BW支持W4A16以及W4A8中都是以marlin算子出现。如下：



名字中包含marlin相关字眼，stage1和stage2均为marlin的算子；
在W8A8量化模型中，由汇编算子或者marlin算子支持，字眼为：moe_w8a8，也分为两个stage。如下：



H20

在H20中，AWQ由marlin提供支持，其他模式由triton提供支持，算子带有fused_moe_kernel字眼。

5.triton_算子

triton_算子为小算子triton优化算子，在BW和H20中均有triton_的字眼，可以将带有triton_字眼的算子集合为一类算子

6.elementwise算子

elementwise算子也有十分明显的elementwise 字眼，可以将其汇聚在一起计算；

7.其他算子

其他算子包括，moe_align_block_size_kernel，moe_sum等，这些算子出现较为单一，直接统计即可

算子耗时拉取

如下图所示，拉取prefill阶段的算子耗时，将pivot table内的数据复制到一个excel表中，对应“算子”，“耗时”，“次数”三列


```

4     sec_match = re.search(r"(\d+)s", s)
5     ms_match = re.search(r"(\d+)ms", s)
6     us_match = re.search(r"(\d+)\u{s}", s)
7     ns_match = re.search(r"(\d+)ns", s)
8     if sec_match:
9         sec = int(sec_match.group(1))
10    if ms_match:
11        ms = int(ms_match.group(1))
12    if us_match:
13        us = int(us_match.group(1))
14    if ns_match:
15        ns = int(ns_match.group(1))
16    return sec * 1000000 + ms * 1000 + us + ns / 1000
17 df['耗时'] = df['耗时'].apply(time_to_us)
18 df.rename(columns={'耗时': '耗时_us'}, inplace=True)

```

作用：将所有算子的耗时统一为us，方便后续计算

融合算子耗时

汇总表格中左右同类型功能的算子耗时，例如

汇总表格中所有带“marlin”命名的算子

汇总表格中所有以“Cijk”开头，以“Squant”结尾的算子

汇总表格中所有以“Cijk”开头，不以“Squant”结尾的算子

汇总表格中所有带“mla”命名的算子

汇总表格中所有带“elementwise”命名的算子

汇总表格中所有带“triton”命名的算子

排序

根据融合后新算子的耗时进行排序，得到如下的输出结果

算子	耗时_us	次数	占比
ncclKernel_AllReduce_Ring_Simple_Sum__b16	3070099.726	2582	24.46%
moe_marlin	2121258.768	1208	16.90%
triton_	1777602.076	14720	14.16%
Cijk_Squant	1429126.778	8967	11.39%
mha	1094694.774	5021	8.72%
Cijk	987973.322	4894	7.87%
void at::native::moe_sum_kernel_shared_mem	691490.554	1217	5.51%
_per_token_quant_int8	500859.924	10185	3.99%
deepseek_scaling_rotary_emb_kernel_gptj	303511.418	2562	2.42%
void at::native::act_and_mul_quant_kernel_opt1	194942.45	1218	1.55%
elementwise	139154.242	6386	1.11%
void at::native::moe_fused_gate_kernel	67173.664	1218	0.54%
concat_kernel_prefill	67782.318	1281	0.54%
Memcpy DtoD (Device -> Device)	42631.454	2541	0.34%
void at::native::moe_align_block_size_kernel_opt	25108.306	1218	0.20%
void at::native::count_and_sort_expert_tokens_kernel	24333.578	1218	0.19%
concat_kernel	5979.51	1220	0.05%
Memcpy DtoH (Device -> Host)	1068.482	21	0.01%
nccl:_all_gather_base	1879.722	21	0.01%
void at::native::reduce_kernel	1229.12	21	0.01%
ncclKernel_AllGather_Ring_Simple_Sum__i8	1624.96	17	0.01%
Memset (Device)	1549.118	2	0.01%
Memcpy HtoD (Host -> Device)	1195.56	126	0.01%
ncclKernel_AllGather_Ring_LL_Sum__i8	254.72	4	0.00%

对比分析：假如得到以下两张性能表格

H20算子占比如下：

deepseek-r1-channel-int8(vllm092) 无mtp TP16 cudagraph in4096 out20 bs21 H20			
prefill 阶段			
算子	耗时_us	次数	占比
fused_moe_kernel	2013774.962	464	27.19%
_per_token_group_quant_fp8	1435514.692	3758	19.39%
gemm_fp8_blockwise	1246160.254	1769	16.83%
ncclDevKernel_AllReduce_Sum_bf16_RING_LL	982694.006	492	13.27%
mha	484115.574	919	6.54%
void at::native::reduce_kernel	351589.067	348	4.75%
triton_	348212.545	3816	4.70%
void at::native::sbtok::gatherTopK	234877.673	696	3.17%
nvjet_tst_128x96_64x7_1x2_h_bz_TNT	100813.336	116	1.36%
void at::native::(anonymous namespace)::CatArrayBatchedC	48354.422	488	0.65%
void at::native::bitonicSortKVInPlace	47700.34	232	0.64%
void vllm::act_and_mul_kernel	29089.179	232	0.39%
nvjet_tst_128x216_64x4_2x1_v_bz_coopA_TNT	25403.114	58	0.34%
void vllm::moe::moe_align_block_size_kernel	18621.647	232	0.25%
void vllm::moe::count_and_sort_expert_tokens_kernel	8619.853	232	0.12%
void vllm::gather_cache<unsigned short>	7304.021	61	0.10%
Memcpy DtoD (Device -> Device)	7279.633	427	0.10%
nvjet_tst_128x320_64x3_2x1_v_bz_coopB_splitK_TNN	6777.27	58	0.09%
void vllm::merge_attn_states_kernel	2396.537	61	0.03%
elementwise	1380.042	859	0.02%
Memset (Device)	747.351	604	0.01%

BW算子耗时占比以及对应同功能算子耗时比较

deepseek-r1-channel-int8(vllm092) 无tbo 无mtp, lightop=1 (BW TP16 cudagraph) V1 0812-beta镜像 BW17 41 in4096 out20 bs21					
prefill 阶段					
初步分析: linear+moe耗时占比: 27.95%, ncd耗时占比33.05%, fp16 gemm+激活量化+moe_sum耗时占比: 12.93%, triton_融合16.07%, mla 4.11%, 共计94.11%					
算子	耗时_us	次数	占比	BW/H20	综合比例
ncclKernel_AllReduce_Ring_Simple_Sum__b16	3261246.991	492	33.03%	0.301324619	
moe_gemm	1910495.31	812	19.35%	1.054059097	
triton_	1586305.375	2794	16.07%	0.219511672	
Cijk_Squant	848658.942	1707	8.60%	1.468387585	
void vllm::moe::moe_sum_kernel	621763.471	406	6.30%	0.565470767	
_per_token_quant_int8	555322.273	2520	5.62%	2.585011914	
mla	405751.232	857	4.11%	1.193133959	
deepseek_scaling_rotary_emb_kernel_gptj	158682.404	488	1.61%		
void vllm::act_and_mul_kernel	135331.398	406	1.37%	0.214947746	
Cijk	99539.87	602	1.01%		
void at::native::(anonymous namespace)::CatArrayBatchedC	83574.298	427	0.85%	0.578580056	
void moe_fused_gate_kernel	70081.217	232	0.71%		
elementwise	68554.465	1464	0.69%	0.020130593	
void vllm::moe::moe_align_block_size_kernel	29878.861	406	0.30%	0.623238182	0.704856279
Memcpy DtoD (Device -> Device)	17619.18	433	0.18%		
void vllm::moe::count_and_sort_expert_tokens_kernel	8181.109	406	0.08%		
Memset (Device)	4729.436	6	0.05%		
Memcpy DtoH (Device -> Host)	5003.958	410	0.05%		
ncclKernel_AllGather_Ring_Simple_Sum__i8	1601.755	3	0.02%		
Memcpy HtoD (Host -> Device)	227.519	24	0.00%		
void at::native::reduce_kernel	235.04	4	0.00%		

查看对应算子BW与H20的算子耗时比，以上为例，triton_/moe_sum明显存在劣势，是性能不足瓶颈所在

特殊说明

每个模型的算子不尽相同，以上是分析deepseek-w4a8模型，主要为moe的相关结构，分析其他模型需要考虑到当前模型的结构和算子，可以先抓取prof进行算子分析，确定需要融合的相关算子名称，再进行融合计算耗时等操作，分析完算子耗时可以得到具体的性能卡点，并针对高占比算子进行优化，提升模型的整体性能

📄 Deepseek 16卡 prof性能分析步骤